SoCET Intro I Project

4-Bit Physical Carry Save Adder

Lisa Bhatkar, Lilja Kiiski, Aditi Mulaka, Tanay Ubale

Project Note!

The team initially mistakenly referred to the 4-Bit CSA as a “Carry-Select Adder” in the initial project proposal. Since then, the team has gone on creating a 4-Bit Carry Save Adder (CSA), and received approval from Jonathon Hong (the course organizer) to continue.

Table of Contents

Technical Section

High-Level Description

Our project aimed to design, analyze, and verify a 4-bit Carry Save Adder which is a digital circuit that computes the sum of three or more binary numbers by utilizing four full adders. The first step was to create a schematic in Logicly to understand the gate structure and the signal flow. Afterwards, we simulated the design in Verilog to ensure the correct outputs. We then implemented the circuit on a breadboard to test its operation which successfully matched our theoretical and simulated outputs.

Hardware & Software

Overview: Software

The team decided to create a SystemVerilog representation of the project pre-creating it physically, in order to emulate what kind of results the team should be getting with the physical CSA.

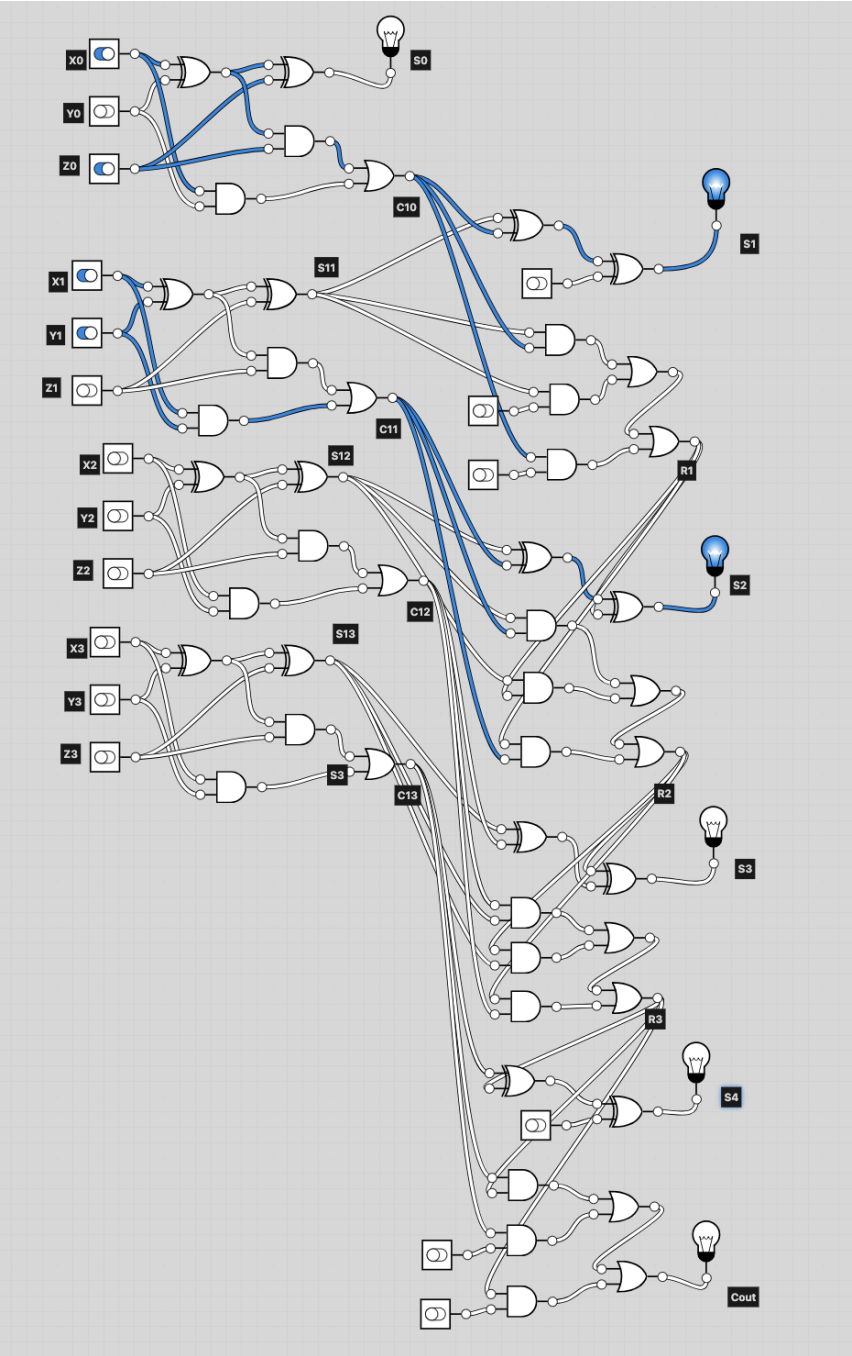
The team used SystemVerilog as the hardware description language, and Makefile to make compilation of the program easier -- as was the standard in class. The SystemVerilog code was split into modules, alongside a testbench to run through all 4096 possibilities of combinations.

The module hierarchy used for the SystemVerilog code is shown below:

```
CSA_4bit
|____Full Adder (x4)
  |____RCA_4bit <- used to sum final sum
    |____Full Adder (x4, reused)
```

Overview: Hardware

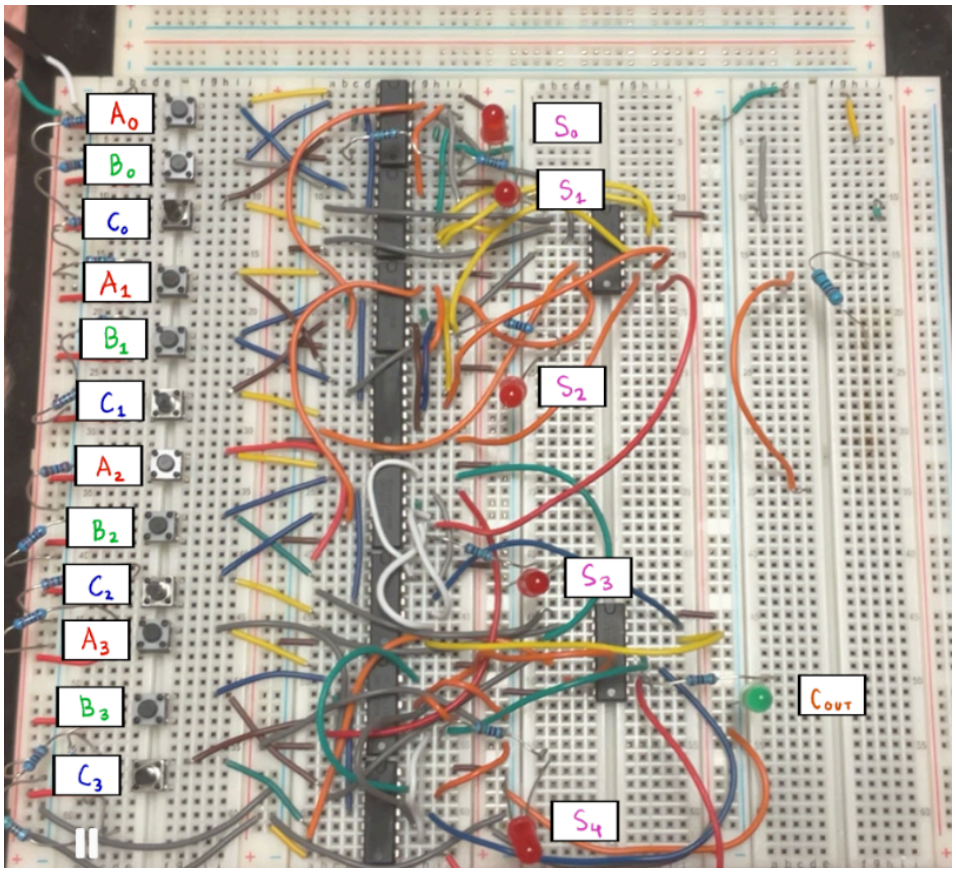
The carry save adder circuit was implemented using a combination of digital logic chips, input devices, and output indicators. Below we have an image showing our entire schematic which we later build on a breadboard.



Parts needed to build carry save adder:

1. Logic Chips - Each of the following chips have 4 gates
    a. Four SN74HC86N Chips - XOR gates
    b. Four SN74HC08N Chips - AND gates
    c. Two CD74HC32E Chips - NOR gates
2. Input Components
    a. 12 Push Buttons – Represented each input as A0, B0, C0, … A3, B3, C3 (seen in image below)
3. Output Components
    a. One Green LED – Represented the final carry-out value (32 place)
    b. Five Red LED - Represented binary for 1, 2, 4, 8, 16 (also seen in image below)
4. Supporting Hardware
    a. Breadboard for circuit assembly
    b. Jumper wires for connections
    c. Resistors
    d. Power supply(3.3V) - To power IC and LED

Our final breadboard is seen below with labels to identify both input and output components. In the long row of IC there are 4 of SN74HC86N chips and SN74HC08N chips where we have most of our full adders. After that we have two CD74HC32E chips which can be seen in the image below as well.



Testing & Benchmarks

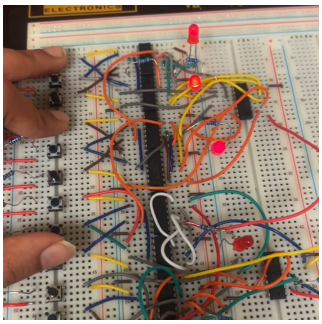
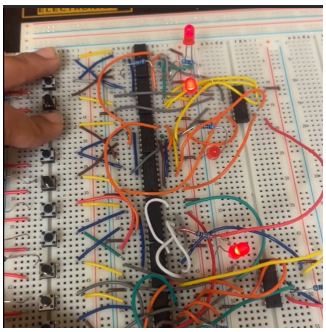
Testing: Hardware

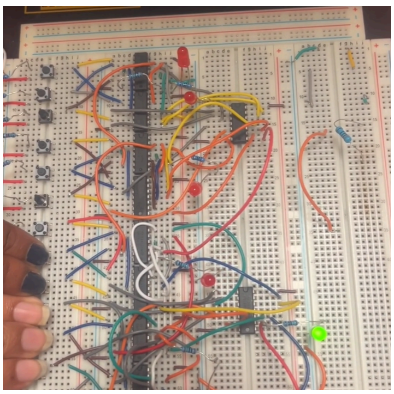
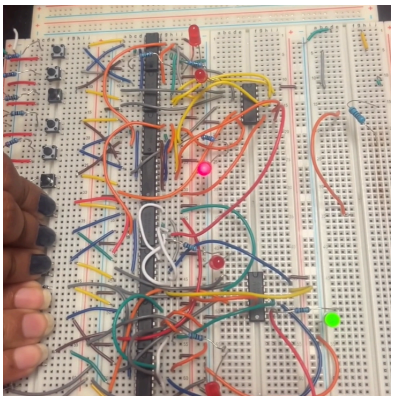
The breadboard consisted of eight full adders. The initial four directly connected to the input push buttons were tested independently with LEDs as output for all eight combinations of three inputs. After these were confirmed to be correct, the final four adders(4-bit ripple carry adder) were constructed, with LED outputs corresponding to the sum and carry of the system.

The first test on the 4-bit CSA ensured all the inputs performed as expected. Each input push button was pressed independently, while all the others were unpressed. The expected result was to be equivalent to the input. After this was fully functioning, the sum of multiple numbers required testing.

The 4-bit CSA has too many input combination possibilities(4096 test cases) to feasibly demonstrate on the breadboard. Therefore, a set of test cases were selected, some by a random number generator and some intentionally chosen, to ensure proper functionality. One notable limitation with this process was the physical constraint of not being able to press certain combinations of buttons by hand at once. This is of less significance considering the thorough testing of the inputs individually and the inclusion of each input button at least once in the selected test cases.

Below are some examples of test cases used to verify functionality:

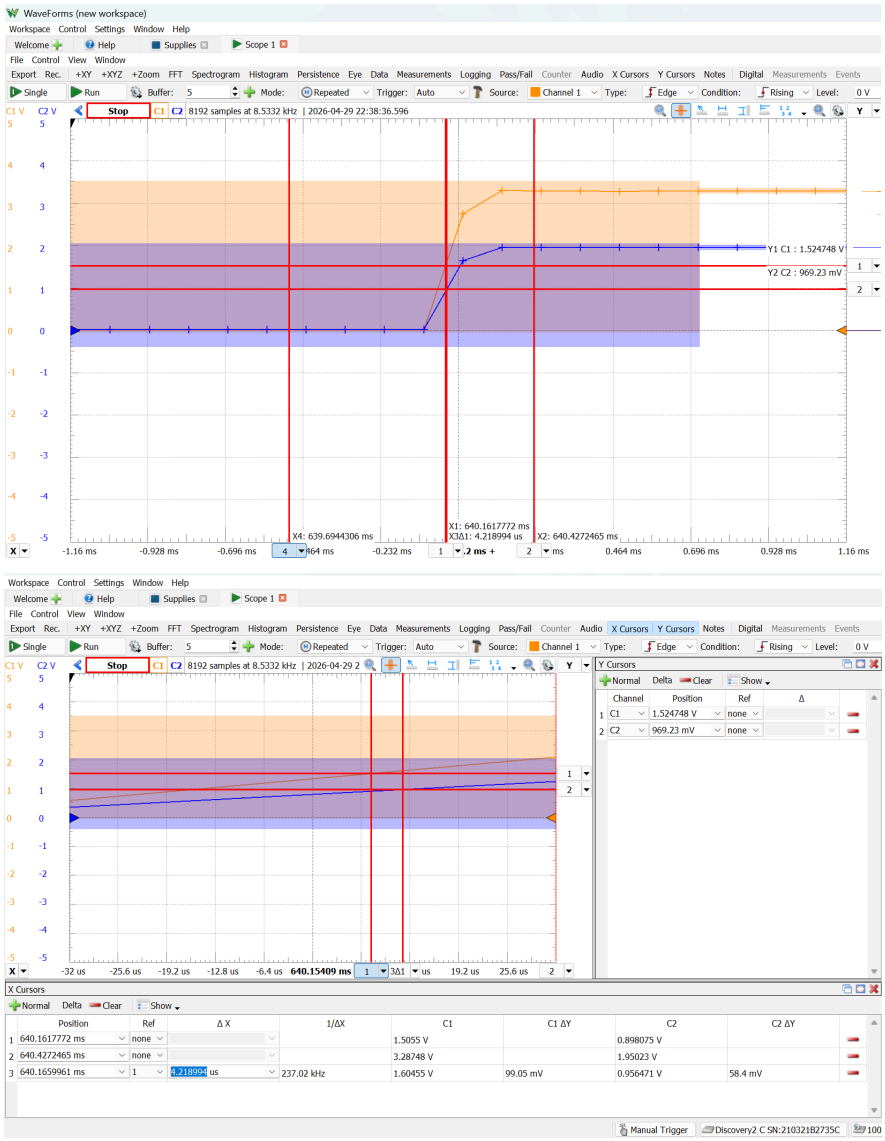
Inputs	Expected Result	Result
A = 0011 = 3 B = 0000 = 0 C = 0100 = 4	S = 00111 C_{OUT} = 0	
A = 0011 = 3 B = 0000 = 0 C = 1000 = 4	S = 01011 C_{OUT} = 0	

A = 1000 = 8 B = 1100 = 12 C = 1100 = 12	S = 00000 C_{OUT} = 1	
A = 1100 = 12 B = 1100 = 12 C = 1100 = 12	S = 00100 C_{OUT} = 1	

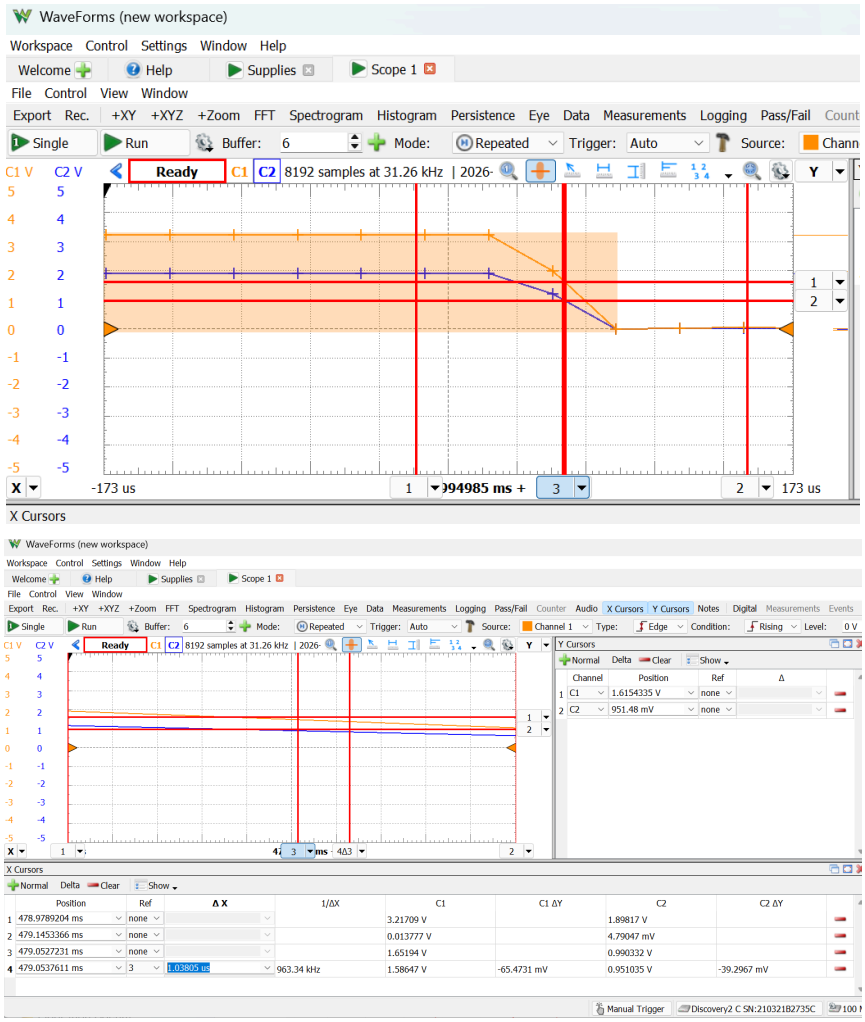
One issue that impacted testing was with the push-buttons used. As these are manually operated, they were prone to falling out of the breadboard during use, resulting in difficulties with conducting test cases. Replacement push buttons were used to attempt to eliminate this issue, and, while there was an improvement, the buttons still fell out of the board frequently.

Finally, propagation delay was measured when the carry bit changes. To measure this, all inputs that were not changing were wired to power or ground directly, without the push-button. The changing input was connected to Channel 1(orange) of the oscilloscope, and the output from the carry was connected to Channel 2(blue). The push-button was used to change the input from low to high and high to low. The rising and falling propagation delays were calculated by measuring the difference in time between the 50% input voltage and 50% output voltage.

Initial inputs	Next inputs(A_3 changes)	Propagation Delay(in reference to A_3
A = 0100 = 4 B = 1100 = 12 C = 1000 = 8 Sum = 24 Carry = 0	A = 1100 B = 1100 C = 1000 Sum = 0 Carry = 1	Rising: 4.218994 us Falling: 1.03805 us



Rising Propagation Delay Oscilloscope Results



Falling Propagation Delay Oscilloscope Results

Testing: Software

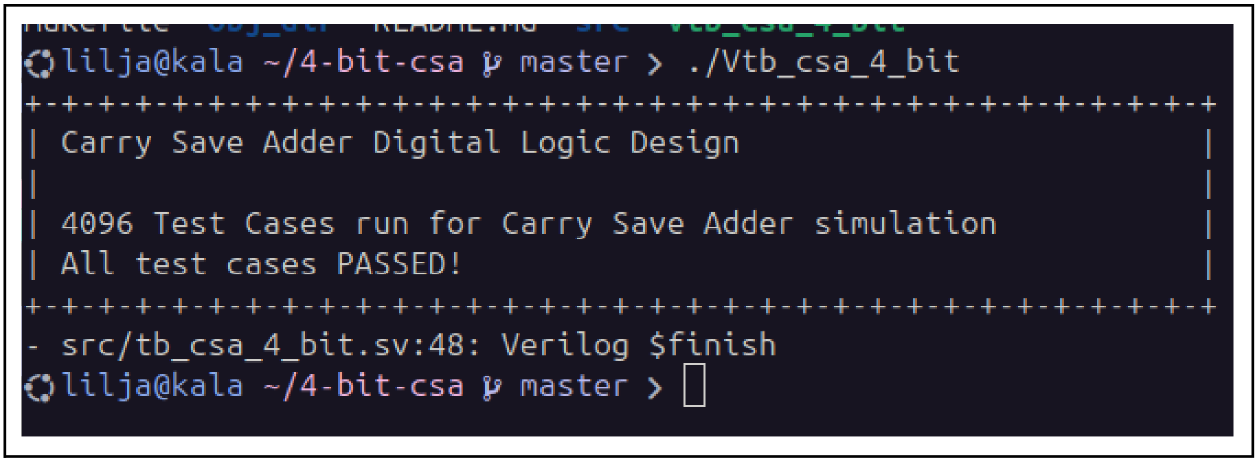
The SystemVerilog code was largely written in one go -- the Full Adder and Ripple Carry Adders based off of labs done previously in class. After implementing the actual CSA itself, and a testbench to run through all the combinations, it came back for testing.

Initially, only a few test cases were run (simply to confirm basic functionality of the CSA). After these test cases ran, a triply nested for loop (a, b, c from 0 to 15) was made to iterate through all the possible combinations of addition that could occur.

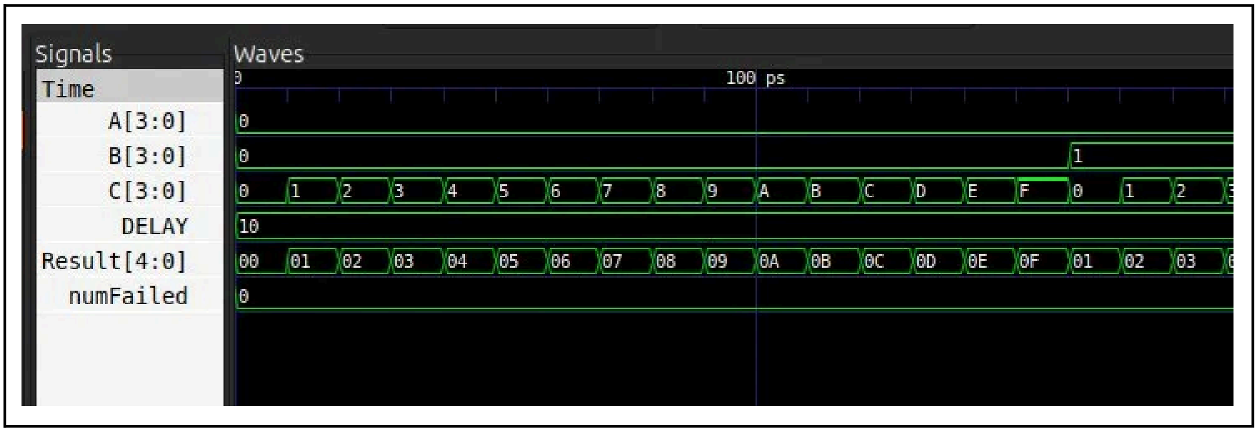
The loop running through all possibilities in the testbench is shown below. The full code is available on GitHub.

```
//Run through 4096 different possible combinations
initial begin
    $dumpfile("obj_dir/waveform.fst");
    $dumpvars(0, tb_csa_4_bit);

    for (int a = 0; a < 16; a++) begin
        for (int b = 0; b < 16; b++) begin
            for (int c = 0; c < 16; c++) begin
                A = 4'(a);
                B = 4'(b);
                C = 4'(c);
                #DELAY; //Delay

                if (Result !== 5'(a + b + c)) begin
                    numFailed++;
                end
            end
        end
    end

    //Display Results! <Not Shown here -- available in GitHub>
end
```

This initially ran with error -- with exactly half of the test cases working (2048/4096 errors). After debugging (found that carries was being used incorrectly), the program finally ran all test cases without error.



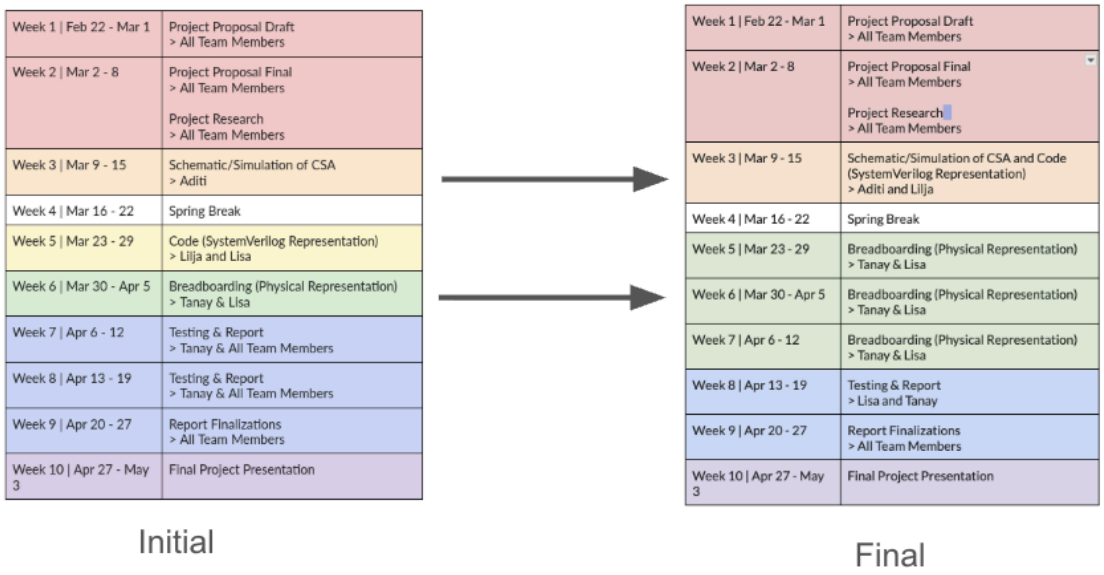
Lastly, the waveform was checked to make sure that the program was actually running as expected.



Division of Labor

The team divided the project into three main components to allow work to be completed as independently as possible: schematic/logical representation which Aditi focused on, SystemVerilog representation which Lilja focused on, and physical representation/breadboarding which Tanay and Lisa focused on. This structure helped streamline progress by assigning clear responsibilities to each member. While the physical representation and breadboarding portion required the most time, the overall workload remained relatively even among the team because we decided to have two people working on it.

Timeline



Week 1 | Feb 22 - Mar 1	Project Proposal Draft > All Team Members
Week 2 | Mar 2 - 8	Project Proposal Final > All Team Members Project Research > All Team Members
Week 3 | Mar 9 - 15	Schematic/Simulation of CSA > Aditi
Week 4 | Mar 16 - 22	Spring Break
Week 5 | Mar 23 - 29	Code (SystemVerilog Representation) > Lilja and Lisa
Week 6 | Mar 30 - Apr 5	Breadboarding (Physical Representation) > Tanay & Lisa
Week 7 | Apr 6 - 12	Testing & Report > Tanay & All Team Members
Week 8 | Apr 13 - 19	Testing & Report > Tanay & All Team Members
Week 9 | Apr 20 - 27	Report Finalizations > All Team Members
Week 10 | Apr 27 - May 3	Final Project Presentation

Initial

Week 1 | Feb 22 - Mar 1	Project Proposal Draft > All Team Members
Week 2 | Mar 2 - 8	Project Proposal Final > All Team Members Project Research > All Team Members
Week 3 | Mar 9 - 15	Schematic/Simulation of CSA and Code (SystemVerilog Representation) > Aditi and Lilja
Week 4 | Mar 16 - 22	Spring Break
Week 5 | Mar 23 - 29	Breadboarding (Physical Representation) > Tanay & Lisa
Week 6 | Mar 30 - Apr 5	Breadboarding (Physical Representation) > Tanay & Lisa
Week 7 | Apr 6 - 12	Breadboarding (Physical Representation) > Tanay & Lisa
Week 8 | Apr 13 - 19	Testing & Report > Lisa and Tanay
Week 9 | Apr 20 - 27	Report Finalizations > All Team Members
Week 10 | Apr 27 - May 3	Final Project Presentation

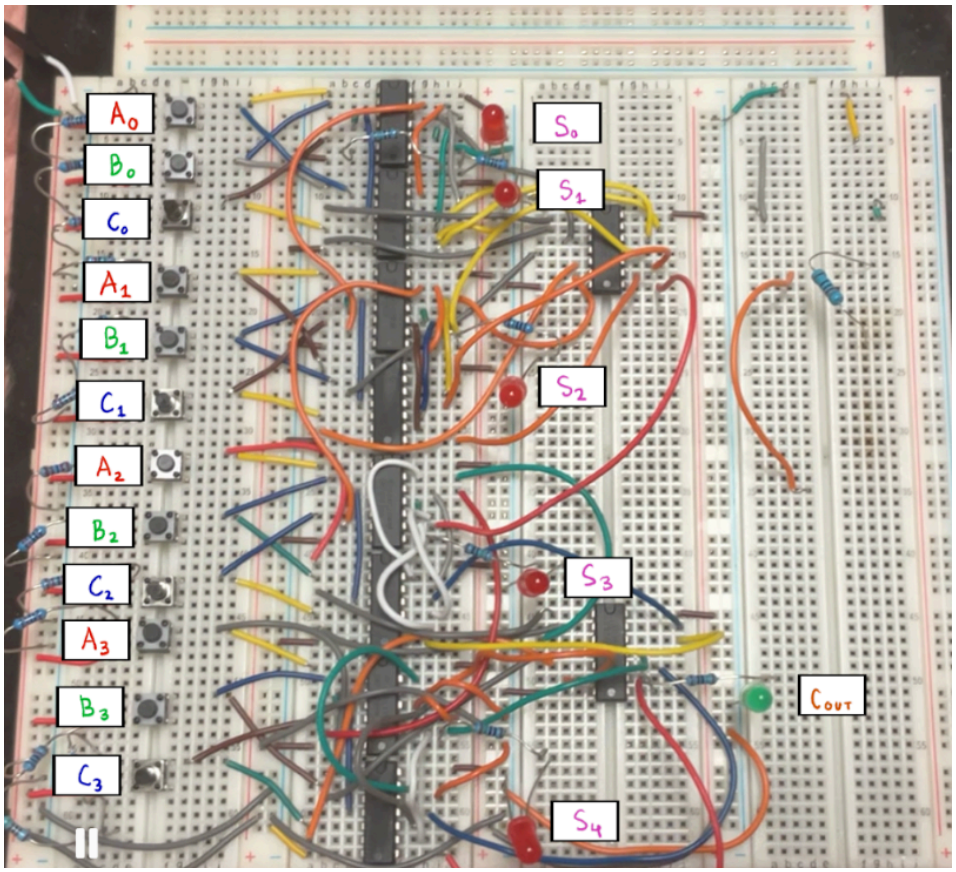
Final

The project timeline was structured over ten weeks which started with drafting and finalizing the project proposal and conducting initial research as a full team. In Week 3, work shifted to more individual assignments such as schematic design, simulation of the carry-save adder, and the code in SystemVerilog. We wanted to finish this before we went off to spring break so when we came back we could focus on the breadboard implementation which spanned multiple weeks and required the most time. Based on progress, the timeline was adjusted to allocate additional time to the physical representation phase, while combining some earlier tasks such as schematic design and coding. The final weeks were dedicated to testing, debugging, and completing the report, leading up to report finalization and the final project presentation. Overall, the timeline evolved to better reflect the complexity of the hardware implementation while still ensuring all milestones were completed.

Operation Documentation

Operation: Hardware

Instructions provided are in reference to the following image:



1. Supply 3.3V to the leftmost power rail, and ground the leftmost ground rail.
2. Determine the three 4-bit numbers to be added, which are inputs A, B, and C.
3. Labeled on the diagram above, press the corresponding push-buttons. The inputs labeled from 0 to 3 are from least to most significant bit.
4. Interpret the results:
    a. Read the red LEDs result for sum, with the topmost S_0 being the LSB and the bottommost S_4 bit being the MSB. If the green LED is off, this is the sum of A, B, and C.
    b. If the green LED is on, add 32 to the number represented by the red LEDs to find the sum.

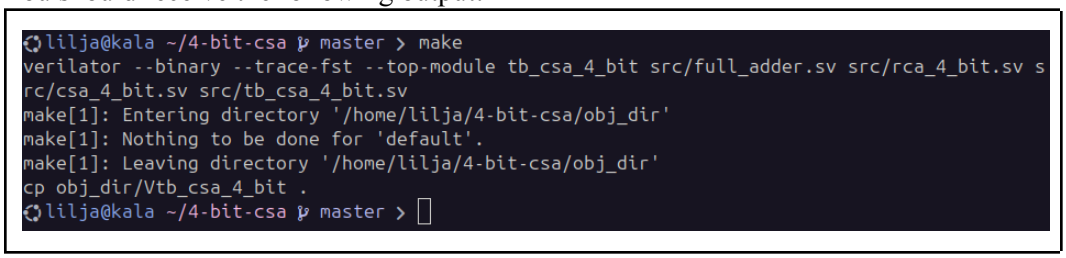
Operation: Software

Operation of the SystemVerilog representation of this project is rather simple

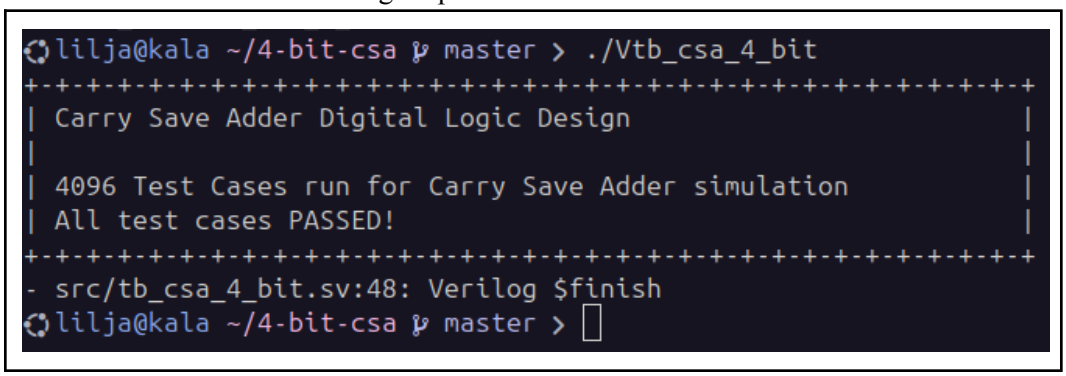
1. Clone the repository, available on GitHub at https://github.com/liljakiiski/4-bit-csa.

2. Make sure to have all necessary libraries installed on your computer
    a. This program is easiest to run on Purdue's ThinLink interface , as it comes with everything required to use Make and run SystemVerilog code
3. Once the repository is cloned, run "make" in your terminal

    a. You should receive the following output:



4. Afterwards, simply run the executable "Vtb_csa_4_bit" by running "./Vtb_csa_4_bit" in the terminal
    a. You should receive the following output:



5. And that's it!

Expansion / Next Steps

The project can be expanded from a 4-bit CSA to a 16-bit design. This expansion will allow the design to handle larger binary numbers and more complex arithmetic operations. To achieve this the design would use sixteen 1-bit full adders and each stage processes one bit from the three inputs. Expanding the project to a 16-bit will make it more complicated to build on a breadboard, providing the opportunity to explore PCB design.

Reflection Section

Learning Outcomes

Lilja: Through this project the team was able to increase their proficiency in SystemVerilog, and overall is now better equipped to create from-scratch SystemVerilog representations for future projects, such as those in ECE 270, 337, and 437. The experience in making this project from scratch will hopefully also help future internship prospects.

Aditi: This project strengthened my understanding of how individual logic gates can work together to achieve arithmetic circuits. By initially building the schematic in Logicly, I better understood how full adders are built using XOR, AND, and OR gates and how the sum and carry are generated. This experience helped me relate the binary addition rules to the output by showing how each logic gate performs the sum and carry.

Tanay: This project really helped my understanding of how logic gates actually work to build real things. I remember in high school learning about them but I never understood how they could actually build something to add different numbers. I think this will be very helpful in the future as well.

Lisa: Although I have learned about circuit building and logic gates through ECE 270, this project was one of the largest I have implemented on a breadboard. With breadboard implementation, there is a lot of potential for errors that can lead to entirely unexpected results. This project taught me the importance of strategically planning the layout of the breadboard such that intermediate testing is possible and changes can be made easily.

Preferences

Lilja: I personally learned that she enjoys the design process involved in creating SystemVerilog representations from scratch, and found the overall process mostly enjoyable. Debugging the code was not pleasant, trying to find the small errors involved, but overall being able to create a project like this from start to finish was fun. She's learned that she would like to pursue learning more about digital design in the future (through ECE 270/337/437).

Aditi: I enjoyed working on the schematic because it allowed me to visually understand the gate logic and how it relates to the binary addition. It was also exciting to try test cases and see the correct outputs since that proved that the design worked as expected. I disliked how if I made small wiring mistakes such as plugging in the output of an OR gate into the wrong AND gate it would completely switch the output. Having to trace the schematic to find the error was time-consuming and confusing.

Tanay: I really enjoyed working on the actual physical breadboard because it gave me the chance to actually see our product working. Seeing it work on a simulation is cool, but getting the chance to build it

with your own hands, test it, and see it working is much more cool. At times it was frustrating to debug, but I thought I learned a lot that I will take away.

Lisa: I found the digital logic implementation really interesting in this project. Currently, I am learning about adders in ECE 270, and with this project I got to see how these adders combine to form a larger system. It made me think about other possible breadboard projects I can design and create when I combine knowledge from this project, ECE 270, and ECE 20008. I liked being able to apply what I learned, and it is always enjoyable to build a circuit and see it work.

Challenges

Lilja: The main challenge faced with the SystemVerilog code was that of debugging. As mentioned previously the code initially ran exactly half of the simulations correctly (2048 errors). Finding the issues at hand took time going through every file (as the issue ended up being two separate things connected with each other), and was overall not pleasant. However, effective debugging skills were learned from this, and greater familiarity with SystemVerilog.

Aditi: One of the biggest challenges I faced was correctly wiring the carry and sum outputs between the stages since it was easy to mix up which signals should connect to which adder. Debugging the schematic required me to test each stage to ensure it was working before connecting it to the next, however, this helped me gain a better understanding of the circuit logic.

Tanay: One of the biggest challenges was debugging. My main part was to build the first 4 full adders and test them all one by one and their inputs. It took a decent amount of time to wire the first one correctly and after I tested that it made the rest a lot easier. However, when I built the 2nd, 3rd, and 4th one with the exact same wiring as the 1st one, the outputs were glitching. I spent a lot of time figuring out exactly where the problem was, which then I found out that the buttons were loose making a switch to a new set of buttons.

Lisa: I constructed the last four full adders and tested the whole 4-bit CSA. I ran into many issues with wiring and the push-buttons being loose, and there were times where I had felt it may have been easier to rebuild those four full adders than diagnose what connection was causing the issue. However, I chose to debug by selecting test cases that would help me narrow down what the issue may be. This was a tedious but successful process. While the wiring could be fixed, the push-buttons continued to have issues with staying in the breadboard especially as they moved often while being pressed. I think toggle switches may be a better substitute for the buttons in future similar processes, as it also simplifies testing with 12 inputs, where it was difficult to press certain combinations of push buttons at once.